

SmartBank v2

Complete Project Handbook

A production-grade digital banking platform built with Flask, vanilla HTML/CSS/JS, SQLite/PostgreSQL, and deployed on Render.

Features: Multi-Role Auth • 2FA • Face Recognition • AI Chatbot • UPI Payments • Crop Marketplace • Encrypted Chat • PDF Statements • Branch Locator

1. Project Overview

SmartBank is a full-stack internet banking system supporting 4 user roles (Customer, Staff, Admin, Agri-Buyer), each with dedicated dashboards. The platform simulates real-world banking operations including account management, fund transfers (NEFT/IMPS/RTGS), card issuance, loan processing, UPI payments, and an agriculture crop marketplace with escrow.

The system is designed with professional software engineering principles: modular Flask Blueprints for separation of concerns, self-healing database migrations for zero-downtime schema evolution, and a triple-fallback email delivery system for reliability.

Feature	Description
Multi-Role Auth	Users, Staff, Admins, Agri-Buyers with dedicated dashboards
2FA Login	Email OTP verification on every login (10min expiry)
Face Authentication	Browser-based face recognition via face-api.js (128-d descriptors)
Core Banking	Accounts, Transactions (NEFT/IMPS/RTGS), Cards, Loans
UPI Payments	Virtual UPI ID creation, UPI PIN, instant transfers
AI Chatbot	Intent-based NLP with 18+ intents and live DB queries
Crop Marketplace	Agriculture marketplace with escrow payments
Staff Portal	KYC verification, loan approvals, card issuance, attendance
Admin Dashboard	System control, user management, audit logs, finances
Mobile Banking	Responsive mobile UI with 4-digit passcode lock
Branch/ATM Locator	Interactive map with 10 seeded Indian branches
Encrypted Chat	AES-128-CBC (Fernet) encrypted live chat
PDF Statements	ReportLab-powered downloadable account statements

2. Technology Stack

Layer	Technology
Backend	Python 3.11, Flask >=3.0, Gunicorn
Database	SQLite (dev) / PostgreSQL via Neon (prod)
Frontend	Vanilla HTML5, CSS3, JavaScript ES6+
Face Auth	face-api.js (TensorFlow.js models)
Encryption	Fernet (AES-128-CBC) via cryptography lib
PDF Engine	ReportLab >=4.2
Email	Brevo API / Resend API / SMTP fallback
Deployment	Render (Web Service + Persistent Disk)
CORS	Flask-CORS >=4.0

3. System Architecture

The application follows a classic 3-tier architecture:

- **Presentation Layer (Frontend):** Static HTML/CSS/JS served by Flask. Device detection auto-routes mobile users to the mobile dashboard.
- **Application Layer (Backend):** Flask app with 9 Blueprints handling all business logic. Core modules provide shared utilities for auth, DB, encryption, and email.
- **Data Layer (Database):** SQLite locally with WAL mode for concurrency; PostgreSQL in production via DATABASE_URL environment variable.

9 Flask Blueprints:

Blueprint	URL Prefix	Purpose
auth_bp	/api/auth	Signup, Login, 2FA, Face-login, Password reset
user_bp	/api/user	Accounts, Transactions, Cards, Loans, UPI
staff_bp	/api/staff	KYC approvals, Loan mgmt, Attendance
admin_bp	/api/admin	System admin, User mgmt, Audit
mobile_bp	/api/mobile	Mobile-specific endpoints
face_bp	/api/face	Face registration and verification
chat_bp	/api/chat	AI chatbot conversations
agri_bp	/api/agri	Agriculture loan applications
marketplace_bp	/api/marketplace	Crop listings, Orders, Escrow

4. Database Schema (25+ Tables)

The database uses 25+ tables organized into 4 groups. All tables use IF NOT EXISTS for resilience. Foreign keys with ON DELETE CASCADE ensure referential integrity.

Core Tables

Table	Purpose	Key Columns
users	Customer accounts	username, email, password, upi_id, face_descriptor
staff	Bank employees	staff_id, department, position, base_salary
admins	System administrators	username, level, face_descriptor
agri_buyers	Marketplace buyers	buyer_id, business_name, gst_number
accounts	Bank accounts	account_number, balance, currency, ifsc
transactions	Transaction log	type, amount, reference_number, mode
cards	Debit/Credit cards	card_number, cvv, credit_limit, status
loans	Loan applications	loan_type, interest_rate, tenure_months

Workflow, Support, Marketplace & Audit Tables

- **card_requests / account_requests / deposit_requests / service_applications** — Pending approval workflows
- **support_tickets / support_messages / live_chat_sessions / live_chat_messages** — Customer support with encrypted chat
- **crop_listings / crop_orders / escrow_transactions / marketplace_chats** — Agriculture marketplace
- **system_audit / user_activity_logs / staff_activity_logs / attendance** — Audit and tracking
- **system_finances** — Bank liquidity fund (seeded at 5 Crore INR)
- **bank_locations** — 10 Indian branch/ATM coordinates
- **notifications / beneficiaries / savings_goals / chat_history** — User features

5. Security Architecture

SmartBank implements defense-in-depth with multiple security layers:

Feature	Implementation
Password Policy	Min 9 chars, uppercase start, digit + special char
Password Hashing	Werkzeug PBKDF2-SHA256
2FA	Email OTP on every login (10min expiry, CSPRNG)
Brute-Force Protection	5 failed attempts = 15-minute IP lockout
Session Security	HttpOnly, SameSite=Lax, Secure, 30min TTL
Face Recognition	128-d descriptor, Euclidean distance < 0.5
RBAC	@login_required + @role_required decorators
Chat Encryption	Fernet AES-128-CBC from SECRET_KEY
Path Traversal	os.path.normpath + base directory check
Audit Logging	All admin/staff actions logged with IP
CORS	Restricted to localhost + *.render.com
Geo-tracking	IP to city/country via ip-api.com

Authentication Flow Explained

1. User enters username + password on the login page.
2. Backend checks IP against brute-force lockout table (5 attempts = 15min ban).
3. Password is verified against the PBKDF2-SHA256 hash stored in the database.
4. If valid, a cryptographically secure 6-digit OTP is generated using `secrets.randbelow()`.
5. OTP is emailed via triple-fallback: Brevo API -> Resend API -> SMTP -> Console log.
6. User submits OTP. Backend verifies it matches and hasn't expired (10min window).
7. Session is created with HttpOnly + SameSite=Lax cookies. TTL is 30 minutes.

6. Face Authentication Deep Dive

SmartBank uses a privacy-preserving approach to face authentication:

Step 1 - Client-Side Detection: The browser loads face-api.js (TensorFlow.js). The user's webcam captures video frames. The library detects the face and extracts a 128-dimensional numerical descriptor (a mathematical fingerprint of facial features). No images are ever sent to the server.

Step 2 - Transmission: Only the 128-element floating-point array is sent via POST to /api/auth/face-login.

Step 3 - Server Comparison: The backend fetches all stored descriptors from the database and calculates the Euclidean distance between the submitted descriptor and each stored one.

Step 4 - Thresholding: If the Euclidean distance is less than 0.5 (strict threshold), the face is considered a match. Values closer to 0 mean higher similarity.

Privacy Benefit: Even if the database is breached, attackers cannot reconstruct the user's face from the 128-d vector. This makes the system GDPR-friendly.

7. AI Chatbot Engine

The chatbot (core/chatbot.py) uses an intent-based NLP approach with 18 intents:

- **Pattern Matching:** Each intent has a list of trigger words. The user's message is lowercased and checked against all patterns.
- **Dynamic Responses:** For balance_inquiry and transaction_history, the bot queries the live database and returns real account data.
- **Auth-Gated Intents:** Sensitive intents (balance, transfers, cards) require the user to be logged in.
- **Why not LLM?** Banking systems must be deterministic. An LLM could hallucinate incorrect balances or interest rates. Pattern-matching guarantees accurate, predictable responses.

Supported intents: greeting, balance_inquiry, transaction_history, transfer_money, loan_inquiry, card_inquiry, support_request, upi_inquiry, fd_inquiry, mutual_fund, insurance, kyc, security, account_opening, branch_atm, offers, crop_marketplace, jokes, who_are_you, thanks, goodbye.

8. Agriculture Ecosystem

GPS-Verified Loans: Farmers apply for agriculture loans by submitting their farm's GPS coordinates (lat/Ing), land size in acres, and crop type. The system stores soil moisture and AI health scores to assist staff in risk assessment. In a production scenario, this would integrate with satellite APIs like ISRO Bhuvan or Sentinel.

Crop Marketplace: A B2B platform connecting farmers (SmartBank account holders) directly with registered agri_buyers. Farmers can list crops with prices, photos, and minimum order quantities. Buyers browse listings and place orders.

Escrow Payments: When a buyer places an order, the payment amount is held in escrow (escrow_transactions table). Funds are only released to the farmer's account once the buyer confirms delivery and quality. This protects both parties from fraud.

Marketplace Chat: Buyers and farmers can communicate about orders through an integrated chat system (marketplace_chats table), enabling price negotiation and delivery coordination.

9. Email Delivery System

SmartBank uses a resilient triple-fallback async email system (core/email_utils.py):

1. **Brevo API** (formerly Sendinblue) - HTTP POST to api.brevo.com/v3/smtp/email
2. **Resend API** - HTTP POST to api.resend.com/emails
3. **SMTP Fallback** - Direct Gmail/custom SMTP with SSL/STARTTLS
4. **Console Fallback** - If ALL methods fail, the OTP is printed to server logs so the user is never locked out during development.

All emails are sent in background threads (threading.Thread with daemon=True) to avoid blocking the HTTP response.

10. Frontend Architecture

Role	Login Page	Dashboard	Size
User	user.html	userdash.html	136KB
Staff	staff.html	staffdash.html	164KB
Admin	(via staff)	admindash.html	162KB
Agri-Buyer	agri-buyer-login.html	agri-buyer-dash.html	35KB
Mobile	mobile-auth.html	mobile-dash.html	87KB

Key JS Files: userdash.js (308KB), staffdash.js (295KB), admindash.js (245KB), mobile-logic.js (234KB), face-api.min.js (664KB), chatbot.js (26KB), signup.js (10KB), api-config.js (7KB).

Device Detection: The system auto-detects mobile vs desktop via device-detector.js and serves the appropriate UI.

11. Deployment on Render

Build: pip install -r requirements.txt

Start: gunicorn backend.main:app --workers 1 --timeout 120

Environment Variables: SECRET_KEY, DATABASE_URL, PYTHON_VERSION=3.11.9, FLASK_ENV=production, RENDER=true

Boot Sequence: Gunicorn imports app -> detects RENDER=true -> connects to PostgreSQL via DATABASE_URL -> runs init_db() + migrate_db() -> loads smart_seed.json -> app is live.

12. Viva Q&A; Cheat Sheet

Q: Why Flask instead of Django?

Flask is a micro-framework giving complete architectural control. I used 9 Flask Blueprints to modularize routes. Django's monolithic approach would have brought unnecessary overhead. Flask allowed me to build custom RBAC middleware and face auth integration exactly as needed.

Q: How is the database secured?

All passwords use PBKDF2-SHA256 hashing. Face data is stored as mathematical descriptors, not images. SQL injection is prevented via parameterized queries (? in SQLite, %s in Postgres). No raw user input is ever interpolated into SQL strings.

Q: How does Face Authentication work?

The browser uses face-api.js (TensorFlow.js) to extract a 128-dimensional facial descriptor from the webcam. Only this numerical array is sent to the server. The backend compares it against stored descriptors using Euclidean distance. A distance < 0.5 means a match. No images are stored, making it GDPR-friendly.

Q: What happens if two people transfer money simultaneously?

SQLite uses WAL (Write-Ahead Logging) mode for concurrent access. In production, PostgreSQL handles strict ACID concurrency with row-level locking. This ensures money is never duplicated or lost during simultaneous writes.

Q: Why not use an LLM for the chatbot?

Banking systems must be deterministic. An LLM could hallucinate incorrect balances or interest rates. Our pattern-matching approach guarantees accurate, predictable responses every time. The chatbot still provides dynamic, live data by querying the database directly.

Q: How does the Agriculture Loan feature work?

Farmers submit GPS coordinates, land size, and crop type. The system stores this with an AI health score for staff-assisted risk assessment. In production, this would integrate with satellite APIs (like ISRO Bhuvan) for soil moisture verification.

Q: Why use external APIs for email instead of just SMTP?

Gmail SMTP is unreliable in production due to spam filters and rate limits. The app uses a triple-fallback: Brevo API -> Resend API -> SMTP -> Console log. This ensures OTPs are always delivered.

Q: Explain the Escrow payment system.

When a buyer orders crops, funds are held in a secure escrow state (escrow_transactions table). Money is only released to the farmer after the buyer confirms delivery and quality. This protects both parties from fraud.

Q: How does the self-healing database work?

The migrate_db() function in core/db.py runs on every startup. It checks if each of the 25+ tables exists and creates missing ones. It then runs 78 column migrations, adding any missing columns. This means the schema evolves without manual SQL or downtime.

Q: What is the Separation of Concerns principle?

Each module has a single responsibility: auth_routes.py handles only authentication, user_routes.py handles only user operations, core/db.py handles only database connections. This makes the code maintainable, testable, and scalable.